

ԼԱԲՈՐԱՏՈՐ ԱՇԽԱՏԱՆՔ 10

Թեմա՝ Ծրագրային ապահովման պաշտպանություն՝ խեղաթյուրման (Obfuscation) ալգորիթմներով

ԲՈՎԱՆԴԱԿՈՒԹՅՈՒՆ

1. Ներածություն
2. Կոդի խեղաթյուրում (Code Obfuscation)
3. խեղաթյուրման հիմնական մեթոդները
4. Obfuscation-ի առավելություններն ու թերությունները
5. Գործնական մաս — Python օրինակներ
6. Գործնական մաս — C++ օրինակներ
7. Advanced Obfuscation
8. Ghidra / IDA ուսումնասիրություն
9. Լրացուցիչ առաջադրանքներ
10. Եզրակացություն

1. ՆԵՐԱԾՈՒԹՅՈՒՆ

Ժամանակակից ծրագրային ապահովման զարգացման ընթացքում մեծ նշանակություն ունի ծրագրերի պաշտպանությունը հակադարձ ինժեներիայից (Reverse Engineering), լիցենզիայի շրջանցումից և վնասաբեր միջամտություններից:

Ծրագրային ապահովման պաշտպանության ամենատարածված մեթոդներից մեկը կոդի խեղաթյուրումն է (Code Obfuscation): Այս տեխնոլոգիան թույլ է տալիս ծրագիրը դարձնել դժվար ընթեռնելի և դժվար վերլուծվող՝ պահպանելով նրա սկզբնական ֆունկցիոնալությունը:

Obfuscation-ը լայն կիրառություն ունի հետևյալ ոլորտներում՝

- Ծրագրային լիցենզավորման համակարգեր

- Բանկային և վճարային ծրագրեր
- Անվտանգության համակարգեր
- Malware պաշտպանություն
- Commercial software protection

Այս լաբորատոր աշխատանքի նպատակն է ուսումնասիրել obfuscation-ի հիմնական մեթոդները, դրանց կիրառման ոլորտները և իրականացնել գործնական օրինակներ Python և C++ լեզուներով:

2. ԿՈՂԻ ԽԵՂԱԹՅՈՒՐՈՒՄ (CODE OBFUSCATION)

2.1 ԻՆչ Է Code Obfuscation-ը

Code Obfuscation-ը ծրագրային պաշտպանության մեթոդ է, որի նպատակն է ծրագրի կոդը դարձնել դժվար հասկանալի մարդու կամ reverse engineering գործիքների համար՝ առանց փոխելու ծրագրի իրական աշխատանքը:

Այս մեթոդը փոխում է ծրագրի կառուցվածքը, տվյալների ներկայացումը կամ կատարման հոսքը այնպես, որ ծրագիրը շարունակի աշխատել նույն կերպ, սակայն նրա վերլուծությունը դառնա բարդ:

2.2 Obfuscation-ի հիմնական նպատակները

✓ Reverse Engineering-ի բարդացում

Ծրագրի իրական ալգորիթմների և տրամաբանության թաքցում:

✓ Intellectual Property Protection

Ծրագրավորողի գաղափարների և ալգորիթմների պաշտպանություն:

✓ License Protection

Crack-երի և լիցենզիայի շրջանցման դժվարացում:

✓ Malware Protection

Վնասաբեր ծրագրերը հաճախ օգտագործում են obfuscation հակավիրուսային համակարգերից թաքնվելու համար:

3. ԽԵՂԱԹՅՈՒՐՄԱՆ ՀԻՄՆԱԿԱՆ ՄԵԹՈԴՆԵՐԸ

3.1 Control Flow Obfuscation

Այս մեթոդը բարդացնում է ծրագրի կատարման հոսքը (execution flow):

Սովորական if/else կառուցվածքները փոխարինվում են state machine-ներով, goto-ներով կամ բարդ switch-case կառուցվածքներով:

Օրինակ

Սովորական տարբերակ՝

```
if(x > 0)
{
    cout << "Positive";
}
else
{
    cout << "Negative";
}
```

Խեղաթյուրված տարբերակ՝

```
int state = 0;
```

```
while(true)
```

```
{
    switch(state)
    {
        case 0:
            if(x > 0)
                state = 1;
            else
```

```

        state = 2;

        break;

    case 1:

        cout << "Positive";

        return 0;

    case 2:

        cout << "Negative";

        return 0;

    }

}

```

Առավելություններ

- Դժվարացնում է CFG (Control Flow Graph) վերլուծությունը
- Խանգարում է decompiler-ներին

3.2 Data Obfuscation

Տվյալները պահվում են կոդավորված կամ փոխակերպված տեսքով:

Օրինակ

```
password = "admin123"
```

Փոխարինվում է՝

```
password = [97,100,109,105,110,49,50,51]
```

Կամ XOR encryption:

Կիրառումներ

- Password hiding
- API key protection

- Sensitive strings hiding
-

3.3 Instruction Substitution

Պարզ հրամանները փոխարինվում են ավելի բարդ համարժեքներով:

Օրինակ

`a = a + b;`

Փոխարինվում է՝

`a = (a ^ b) + 2 * (a & b);`

Այս արտահայտությունը տալիս է նույն արդյունքը, սակայն ավելի դժվար է հասկանալ:

3.4 Dead Code Injection

Ծրագրում ավելացվում է կոդ, որը երբեք չի կատարվում:

Օրինակ

`if False:`

`print("Never executed")`

կամ՝

`for(int i=0;i<100000;i++)`

`{`

`int t = i * 777;`

`}`

Նպատակ

- Reverse engineering-ի բարդացում
 - Static analysis-ի շփոթեցում
-

3.5 Code Flattening

Բոլոր ֆունկցիաները միավորվում են մեկ dispatcher-ի մեջ:

Այս մեթոդը հաճախ օգտագործվում է malware-ներում:

Առանձնահատկություններ

- State machine architecture
 - Switch-case dispatcher
 - Flattened control flow
-

3.6 String Encoding / Encryption

String-երը պահվում են encrypted կամ encoded ձևով:

Օրինակ

"hello"

դառնում է՝

[104,101,108,108,111]

կամ XOR encoded array:

3.7 Opaque Predicates

Ստեղծվում են պայմաններ, որոնք միշտ նույն արդյունքն են տալիս, սակայն դժվար են վերլուծվում:

Օրինակ

if((x*x) >= 0)

Այս պայմանը միշտ TRUE է:

Նպատակ

- Control flow-ի խճճում
 - Static analyzer-ների խաբում
-

4. OBFUSCATION-Ի ԱՌԱՎԵԼՈՒԹՅՈՒՆՆԵՐՆ ՈՒ ԹԵՐՈՒԹՅՈՒՆՆԵՐԸ

Անվտանգության բարձրացում

Ծրագիրը դառնում է դժվար reverse engineering-ի համար:

Ալգորիթմների պաշտպանություն

Հնարավոր է թաքցնել բիզնես տրամաբանությունը:

License Protection

Դժվարացնում է crack-երի ստեղծումը:

Malware Analysis Prevention

Վնասաբեր ծրագրերը դժվար է վերլուծել:

Կոդի բարդացում

Ծրագիրը դժվար է debug անել և սպասարկել:

Performance Overhead

Որոշ obfuscation տեխնիկաներ դանդաղեցնում են ծրագիրը:

Maintenance Difficulty

Developer-ների համար բարդանում է ծրագրի զարգացումը:

5. ԳՈՐԾՆԱԿԱՆ ՄԱՍ — PYTHON ՕՐԻՆԱԿՆԵՐ

ՕՐԻՆԱԿ 1 — XOR STRING OBFUSCATION

```
# XOR Encryption Function
```

```
def xor_encrypt(text, key):
```

```
    encrypted = []
```

```
for ch in text:
    encrypted.append(ord(ch) ^ key)

return encrypted
```

XOR Decryption Function

```
def xor_decrypt(data, key):
```

```
    decrypted = ""
```

```
    for num in data:
```

```
        decrypted += chr(num ^ key)
```

```
    return decrypted
```

Main Program

```
text = "Hello World"
```

```
key = 25
```

```
encrypted = xor_encrypt(text, key)
```

```
print("Encrypted Data:")
```



```
print(encrypted)
```

```
decrypted = xor_decrypt(encrypted, key)
```

```
print("\nDecrypted Data:")
```

```
print(decrypted)
```

Կոդի բացատրություն

xor_encrypt()

Յուրաքանչյուր սիմվոլ XOR է արվում key-ի հետ:

xor_decrypt()

Նույն key-ով տվյալը վերականգնվում է:

Արդյունք

Encrypted Data:

[81, 124, 117, 117, 118, 57, 78, 118, 107, 117, 125]

Decrypted Data:

Hello World

ՕՐԻՆԱԿ 2 — CONTROL FLOW + DEAD CODE

```
def protected_function():
```

```
    state = 0
```

```
    while True:
```

```
if state == 0:
```

```
    x = 5
```

```
    # Dead Code
```

```
    for i in range(1000):
```

```
        temp = i * 999
```

```
    state = 1
```

```
elif state == 1:
```

```
    y = 10
```

```
    state = 2
```

```
elif state == 2:
```

```
    result = x + y
```

```
    print("Result =", result)
```

```
    break
```

```
protected_function()
```

Բացատրություն

Այս օրինակում՝

- Կա state machine
 - Կա dead code loop
 - Control flow-ը բարդացված է
 - Reverse engineering-ը դժվարանում է
-

```
import xor_obfuscation
```

```
import control_flow
```

```
print("Program executed successfully")
```

6. ԳՈՐԾՆԱԿԱԼ ՄԱՍ — C++ ՕՐԻՆԱԿՆԵՐ

XOR STRING OBFUSCATION

```
#include <iostream>
```

```
#include <vector>
```

```
using namespace std;
```

```
vector<int> encrypt(string text, int key)
```

```
{
```

```
    vector<int> result;
```

```
    for(char c : text)
    {
        result.push_back(c ^ key);
    }

    return result;
}

string decrypt(vector<int> data, int key)
{
    string result = "";

    for(int x : data)
    {
        result += char(x ^ key);
    }

    return result;
}

int main()
{
    string text = "SecretPassword";

    int key = 7;
```

```
vector<int> encrypted = encrypt(text, key);

cout << "Encrypted: ";

for(int x : encrypted)
{
    cout << x << " ";
}

cout << endl;

cout << "Decrypted: ";
cout << decrypt(encrypted, key);

return 0;
}
```

Արդյունք

Encrypted:

84 98 100 117 98 115 87 102 116 116 112 104 117 99

Decrypted:

SecretPassword

CONTROL FLOW OBFUSCATION

```
#include <iostream>
```

```
using namespace std;
```

```
int main()
```

```
{
```

```
    int state = 0;
```

```
    int x = 0;
```

```
    int y = 0;
```

```
    while(true)
```

```
    {
```

```
        switch(state)
```

```
        {
```

```
            case 0:
```

```
                x = 5;
```

```
                // Dead Code
```

```
                for(int i=0;i<10000;i++)
```

```
                {
```

```
                    int temp = i * 123;
```

```
                }
```

```
        state = 1;
        break;

    case 1:

        y = 10;

        state = 2;
        break;

    case 2:

        cout << "Result = " << x + y << endl;
        return 0;
    }
}

return 0;
}
```

7. ADVANCED OBFUSCATION

```
#include <iostream>
```

```
#include <windows.h>
```

```
using namespace std;
```

```
bool anti_debug()
```

```
{
```

```
    return IsDebuggerPresent();
```

```
}
```

```
int main()
```

```
{
```

```
    if(anti_debug())
```

```
    {
```

```
        cout << "Debugger detected!" << endl;
```

```
        return 0;
```

```
    }
```

```
int state = 0;
```

```
int value = 0;
```

```
while(true)
```

```
{
```

```
    switch(state)
```

```
    {
```

```
        case 0:
```

```
            value = (5 ^ 2) + 2 * (5 & 2);
```

```
            state = 1;
```

```
            break;
```


case 1:

```
// Dead code
for(int i=0;i<5000;i++)
{
    int temp = i * 77;
}
```

```
state = 2;
```

```
break;
```

case 2:

```
cout << "Protected Value = " << value << endl;
```

```
return 0;
```

```
}
```

```
}
```

```
return 0;
```

```
}
```

Բացատրություն

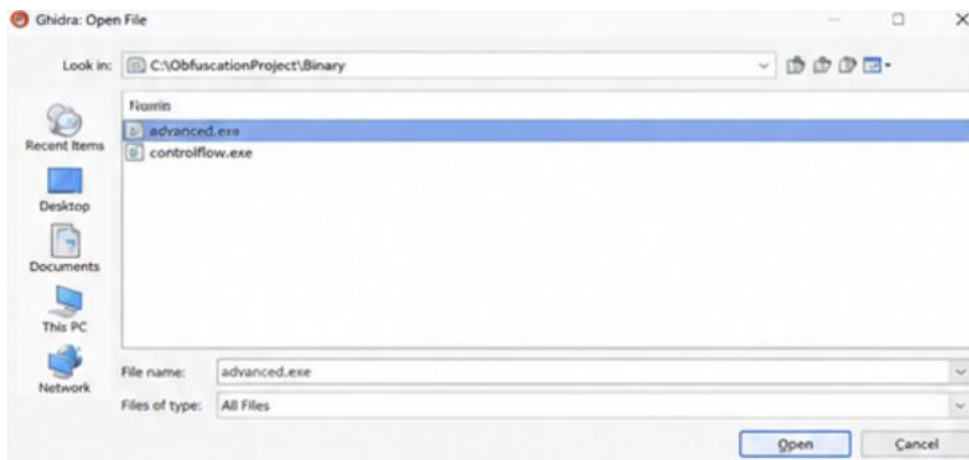
Այս օրինակում կիրառված են՝

- Anti-debugging
- Instruction substitution

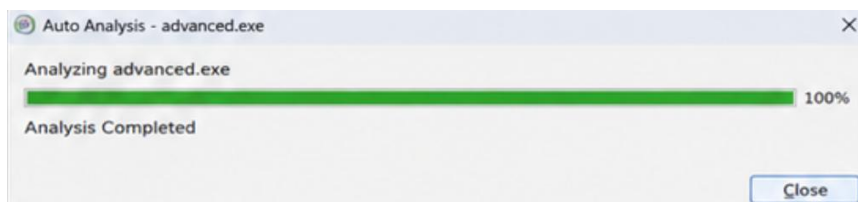
- Dead code injection
- Control flow obfuscation

8. GHIDRA / IDA

1. Ծրագրի բացում



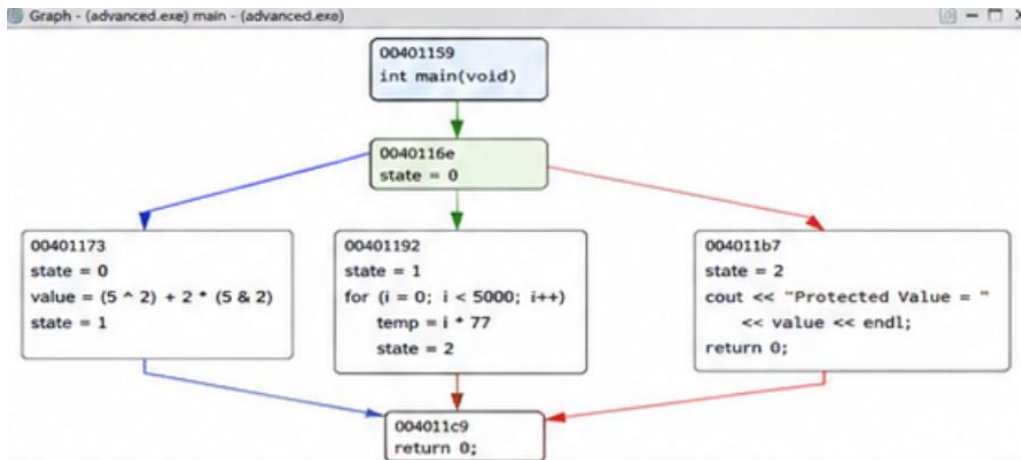
2. Auto Analysis



3. Functions Window

Function Name	Entry Point
_init	00401000
_start	00401030
_dl_relocate_static_pie	00401060
deregister_tm_clones	004010a0
register_tm_clones	004010d0
__do_global_ctors_aux	00401100
frame_dummy	00401140
main	00401150
__libc_csu_init	004011c4
__libc_csu_fini	0040122b
__term_proc	00401234

4. Control Flow Graph



5. Decompiled Code

```
1 int main(void)
2 {
3     int state;
4     int i;
5     int value;
6
7     state = 0;
8     while(true) {
9         switch(state) {
10             case 0:
11                 value = (5 ^ 2) + 2 * (5 & 2);
12                 state = 1;
13                 break;
14             case 1:
15                 for (i = 0; i < 5000; i = i + 1) {
16                     int temp = i * 77;
17                 }
18                 state = 2;
19                 break;
20             case 2:
21                 std::cout << "Protected Value = " << value << std::endl;
22                 return 0;
23         }
24     }
25 }
```

9. ԼՐԱՑՈՒՑԻՉ ԱՌԱՋԱԴՐԱՆՔՆԵՐ

9.1 Polymorphic Obfuscation

Polymorphic obfuscation-ի դեպքում ծրագիրը ամեն գործարկման ժամանակ փոխում է իր կառուցվածքը:

Առանձնահատկություններ

- Signature detection-ի դժվարացում
- Malware-ում լայն կիրառություն

9.2 Metamorphic Obfuscation

Metamorphic malware-ը ամբողջությամբ փոխում է իր կոդը՝ պահպանելով ֆունկցիոնալությունը:

Օգտագործվող մեթոդներ

- Instruction substitution
- Register renaming
- Code reordering

9.3 Anti-Debugging Techniques

Windows API

IsDebuggerPresent()

Linux API

ptrace()

Նպատակ

- Debugger-ի հայտնաբերում
- Dynamic analysis-ի կանխում

9.4 Simple Executable Packer

Packer-ը executable ֆայլը compress/encrypt է անում և runtime-ում unpack է անում:

Օգտագործման ոլորտներ

- Software protection
- Malware hiding
- License systems

9.5 Crackme Example

Crackme-ը փոքր ծրագիր է, որի նպատակն է reverse engineering-ի միջոցով շրջանցել password verification-ը:

Օրինակ

```
if(password == "admin123")
{
    cout << "Access Granted";
}
else
{
    cout << "Access Denied";
}
```

Reverse engineer-ի նպատակն է bypass անել ստուգումը:

10. ԵԶՐԱԿԱՑՈՒԹՅՈՒՆ

Այս լաբորատոր աշխատանքի ընթացքում ուսումնասիրվեցին ծրագրային ապահովման պաշտպանության հիմնական մեթոդները՝ օգտագործելով obfuscation ալգորիթմներ:

Աշխատանքի ընթացքում իրականացվեցին հետևյալ թեմաները՝

- XOR String Obfuscation
- Control Flow Obfuscation
- Dead Code Injection
- Instruction Substitution
- Anti-Debugging Techniques
- Ghidra / IDA ուսումնասիրություն
- Advanced Obfuscation

Գործնական մասում ստեղծվեցին Python և C++ ծրագրեր, որոնք ցույց տվեցին obfuscation տեխնիկաների իրական կիրառումը: